Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Jointures (INNER JOIN, LEFT JOIN) et Agrégation (GROUP BY, HAVING)

Synthèse exécutive : Ce cours aborde la construction et la logique d'exécution des requêtes SQL complexes sur des bases de données relationnelles normalisées. Il détaille l'association de données réparties dans plusieurs tables grâce aux jointures relationnelles (INNER JOIN et LEFT JOIN) en évitant les pièges de produits cartésiens involontaires, ainsi que la structuration des données en groupes statistiques au moyen de GROUP BY et le filtrage des agrégats via HAVING. L'assimilation du cycle d'exécution séquentiel d'une requête SQL par le SGBD (FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY) est le fil conducteur indispensable pour réussir l'UE R1.05.

Les Jointures Relationnelles : INNER JOIN vs LEFT JOIN et prévention du Produit Cartésien

Dans un SGBDR (Système de Gestion de Base de Données Relationnel), la normalisation impose la dispersion contrôlée des données entre plusieurs tables. La jointure est l'opération algébrique indispensable pour réassembler ces informations au moment de l'interrogation. La clause JOIN est évaluée au tout début de l'exécution dans le bloc FROM. Il est obligatoire de spécifier le prédicat de liaison (ex: ON E.id_etud = N.id_etud) reliant la clé étrangère d'une table à la clé primaire de l'autre table. Sans cette clause ON, le moteur SQL applique un produit cartésien, combinant chaque enregistrement de la première table avec l'ensemble des enregistrements de la seconde table, risquant d'entraîner le plantage ou le surcoût mémoire du serveur. L'INNER JOIN filtre le résultat en ne conservant que les correspondances effectives, tandis que le LEFT JOIN garantit que les enregistrements de la table de gauche apparaissent tous dans le résultat final, en injectant la valeur NULL pour les attributs de droite en l'absence de correspondance.

- Normalisation relationnelle : Séparation des entités métiers dans des tables distinctes reliées par Clé Primaire (PK) et Clé Étrangère (FK) pour éliminer les redondances.
- INNER JOIN (Jointure Interne) : Opération d'intersection restituant exclusivement les paires de n-uplets satisfaisant la condition d'équivalent déclarée dans la clause ON.
- LEFT JOIN (Jointure Externe Gauche) : Conservation de l'exhaustivité de la table de gauche, complétée par des valeurs NULL si aucune correspondance n'existe dans la table de droite.
- Produit Cartésien (CROSS JOIN) : Évaluation catastrophique causée par l'omission du critère de jointure (clause ON), générant un croisement combinatoire N x M exhaustif et très coûteux en performances.

Cas concret enseignant : *Le professeur compare l'INNER JOIN à l'accès sécurisé par portique au campus de l'IUT de Bayonne : seuls les étudiants possédant une carte valide ET une inscription effective peuvent franchir la porte. Si une correspondance manque d'un côté, la ligne est éliminée. Pour le LEFT JOIN, l'analogie est la liste d'appel globale du secrétariat de l'IUT : tous les étudiants inscrits apparaissent obligatoirement sur le registre ; si un étudiant n'a pas encore de note saisie, son bulletin affiche la valeur 'NULL'.*

Regroupement de Données et Agrégation Statistiques : Clauses GROUP BY et HAVING

L'agrégation permet de réaliser des synthèses décisionnelles sur des données. Dès que la clause GROUP BY est invoquée, la table initiale est scindée en partitions indépendantes. Une erreur conceptuelle fréquente en partiel consiste à essayer de filtrer un calcul statistique dans la clause WHERE (par exemple `WHERE AVG(note) > 10`). Cela est techniquement impossible : la clause WHERE s'exécute sur les lignes individuelles de la table source AVANT que les groupes ne soient constitués et que les fonctions agrégées ne soient calculées. SQL fournit la clause HAVING pour appliquer des critères de restriction postérieurs au regroupement. De plus, il est formellement

interdit d'insérer dans la clause SELECT un attribut non agrégé s'il n'est pas déclaré dans le GROUP BY, sous peine d'erreur sémantique SQL.

• Fonctions d'agrégation (COUNT, SUM, AVG, MIN, MAX) : Opérations statistiques réduisant un ensemble de n-uplets à une valeur scalaire unique.

• Clause GROUP BY : Partitionnement de la table filtrée en sous-groupes homogènes partageant la même valeur sur les attributs spécifiés.

• Règle d'or du SELECT avec GROUP BY : Toute colonne présente dans le SELECT doit obligatoirement figurer dans le GROUP BY ou être enveloppée dans une fonction d'agrégation.

• Clause HAVING : Prédicat de filtrage s'appliquant exclusivement après le regroupement sur les sous-groupes formés et les résultats agrégés.

• Ordre logique d'exécution SQL : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant illustre le fonctionnement avec le dépouillement physique des copies d'examen dans l'amphithéâtre de l'IUT : 1. GROUP BY : Le professeur répartit physiquement toutes les copies dans trois bannettes distinctes selon le département (Informatique, GEA, TC). 2. Agrégation (AVG) : Pour chaque bannette, il calcule la moyenne générale. 3. HAVING : Si le directeur exige de ne conserver que les paquets dont la moyenne dépasse 12/20, c'est la clause HAVING qui élimine les bannettes non conformes. La clause WHERE aurait servi avant le tri à écarter les copies des étudiants fraudeurs.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : C'est une impossibilité liée à l'ordre d'exécution logique du SGBD. La clause WHERE intervient en étape 2 pour filtrer les lignes/tuples individuels de la table source. À ce stade, la phase de regroupement (GROUP BY) n'a pas encore eu lieu, la fonction AVG(note) n'existe donc pas encore. Pour filtrer sur le résultat d'un agrégat, il faut utiliser la clause HAVING, qui s'exécute après le regroupement (étape 4).

Question étudiant : Que se passe-t-il si j'oublie d'inclure dans la clause GROUP BY une colonne présente dans le SELECT ?
Réponse du professeur : Le SGBD lève immédiatement une erreur de syntaxe ou de sémantique (ex: 'not a GROUP BY expression'). Si vous demandez d'afficher un nom d'étudiant sans le mettre dans le GROUP BY alors qu'il y a un regroupement par promo, le système ne sait pas quel nom choisir parmi la cinquantaine d'étudiants d'une même promo. Soit la colonne fait partie du critère de regroupement (GROUP BY), soit elle doit être synthétisée par une fonction d'agrégation (COUNT, MAX, etc.).

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle de l'IUT composée des deux tables : - Etudiant(id_etud, nom, id_promo) - Note(id_note, id_etud, matiere, note) Écrire la requête SQL permettant d'afficher le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants possédant au moins 2 notes enregistrées dans cette matière et une moyenne stricte supérieure à 12/20. Le résultat doit être trié par moyenne décroissante.

Démarche de résolution :
Étapes méthodologiques de construction pas à pas : 1. Assemblage des tables (FROM / JOIN) : On réalise une équijointure interne entre Etudiant (alias E) et Note (alias N) sur le champ pivot id_etud via 'FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud'. 2. Filtrage des lignes individuelles (WHERE) : On restreint les notes consultées à la matière 'R1.05' avant tout calcul avec 'WHERE N.matiere = 'R1.05''. 3. Partitionnement en groupes (GROUP BY) : On regroupe les enregistrements par étudiant en précisant 'GROUP BY E.id_etud, E.nom'. (L'ajout de E.nom est obligatoire pour respecter la règle de projection du SELECT). 4. Filtrage des groupes (HAVING) : On exprime les deux conditions agrégées requises : avoir au moins 2 notes (COUNT(N.id_note) >= 2) ET une moyenne > 12 (AVG(N.note) > 12) via la clause 'HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12'. 5. Projection des attributs (SELECT) : On extrait le nom de l'étudiant et le calcul renommé 'SELECT E.nom, AVG(N.note) AS moyenne'. 6. Tri des résultats (ORDER BY) : On trie le jeu final de données par ordre décroissant

avec 'ORDER BY moyenne DESC'.

```
SELECT E.nom, AVG(N.note) AS moyenne
FROM Etudiant E
INNER JOIN Note N ON E.id_etud = N.id_etud
WHERE N.matiere = 'R1.05'
GROUP BY E.id_etud, E.nom
HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12
ORDER BY moyenne DESC;
```

Résultat : La requête renvoie un tableau contenant le nom des étudiants qualifiés et leur moyenne calculée en R1.05 pour ceux qui satisfont les critères d'agrégation (au moins 2 notes et moyenne > 12), triés du meilleur au moins bon.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING dans une requête SQL ?

R : La clause WHERE filtre les tuples individuels avant la phase de regroupement et ne peut pas évaluer de fonctions d'agrégation. La clause HAVING s'exécute après la clause GROUP BY et filtre spécifiquement les groupes créés en évaluant des agrégats (AVG, COUNT, SUM, etc.).

Q : Quel est le comportement d'un LEFT JOIN si un enregistrement de la table de gauche ne possède aucune correspondance dans la table de droite ?

R : L'enregistrement de la table de gauche est conservé dans le résultat final, et tous les attributs issus de la table de droite reçoivent la valeur spéciale 'NULL'.

Q : Si la table 'Etudiant' contient 10 enregistrements et la table 'Note' contient 5 enregistrements, combien de lignes retourne un produit cartésien (CROSS JOIN) entre ces deux tables ?

R : Un produit cartésien combine chaque ligne de la première table avec toutes les lignes de la seconde table, retournant exactement 10 x 5 = 50 lignes.